

Towards Free Lunch Derandomization from Necessary Assumptions (And OWFs)

Marshall Ball  

New York University, NY, USA

Lijie Chen  

University of California, Berkeley, CA, USA

Roei Tell  

University of Toronto, Canada

Abstract

The question of optimal derandomization, introduced by Doron et. al (JACM 2022), garnered significant recent attention. Works in recent years showed conditional superfast derandomization algorithms, as well as conditional impossibility results, and barriers for obtaining superfast derandomization using certain black-box techniques.

Of particular interest is the extreme high-end, which focuses on “free lunch” derandomization, as suggested by Chen and Tell (FOCS 2021). This is derandomization that incurs essentially no time overhead, and errs only on inputs that are infeasible to find. Constructing such algorithms is challenging, and so far there have not been any results following the one in their initial work. In their result, their algorithm is essentially the classical Nisan-Wigderson generator, and they relied on an ad-hoc assumption asserting the existence of a function that is non-batch-computable over all polynomial-time samplable distributions.

In this work we deduce free lunch derandomization from a variety of natural hardness assumptions. In particular, we do not resort to non-batch-computability, and the common denominator for all of our assumptions is hardness over all polynomial-time samplable distributions, which is necessary for the conclusion. The main technical components in our proofs are constructions of new and superfast targeted generators, which completely eliminate the time overheads that are inherent to all previously known constructions. In particular, we present an alternative construction for the targeted generator by Chen and Tell (FOCS 2021), which is faster than the original construction, and also more natural and technically intuitive.

These contributions significantly strengthen the evidence for the possibility of free lunch derandomization, distill the required assumptions for such a result, and provide the first set of dedicated technical tools that are useful for studying the question.

2012 ACM Subject Classification Theory of computation → Computational complexity and cryptography; Theory of computation → Pseudorandomness and derandomization

Keywords and phrases Pseudorandomness, Derandomization

Digital Object Identifier 10.4230/LIPIcs.CCC.2025.31

Related Version *Full Version:* <https://eccc.weizmann.ac.il/report/2025/010/>

Funding *Marshall Ball:* Marshall Ball is supported in part by a Simons Junior Faculty Fellowship. *Lijie Chen* is supported by a Miller Research Fellowship.

Roei Tell: Roei Tell is supported by the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grant RGPIN-2024-04490.

Acknowledgements We thank Alec Dewulf for useful comments about Section 2, and for pointing out several inaccuracies. Part of this work was performed while the authors were visiting the Simons Institute for the Theory of Computing at UC Berkeley.

 © Marshall Ball, Lijie Chen, and Roei Tell;
licensed under Creative Commons License CC-BY 4.0
40th Computational Complexity Conference (CCC 2025).

Editor: Srikanth Srinivasan; Article No. 31; pp. 31:1–31:20
Leibniz International Proceedings in Informatics

 **LIPICs** Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

 **COMPUTATIONAL
COMPLEXITY
CONFERENCE**

1 Introduction

The classical $pr\mathcal{BPP} = pr\mathcal{P}$ conjecture asserts that all randomized algorithms solving decision problems can be efficiently simulated by deterministic algorithms, with only a polynomial time overhead. Celebrated hardness vs randomness results in complexity theory conditionally yield such derandomization, while incurring an overhead that is polynomial but large [25, 28, 34, 37]. Can we go further than that, simulating randomness with smaller overhead?

The challenge of finding the minimal overhead for derandomization was introduced by Doron et al. [17], and has been intensively studied since. The goal in this context is to construct “superfast” derandomization algorithms, with small overhead and ideally with no overhead at all, under the weakest possible hardness assumption. Among the known results are conditional superfast derandomization algorithms (see, e.g., [17, 12, 11]), conditional impossibility results (see [11, 14]), barriers for certain black-box techniques (see [35]), and a study of this question in the setting of interactive proof systems (see [14]) and in the space-bounded setting (see [19, 18]).

When focusing on superfast derandomization that succeeds on all inputs (i.e., in the worst-case), a potential emerging picture is starting to seem clearer. Following [17], Chen and Tell [12] showed that assuming one-way functions and sufficiently strong lower bounds for non-uniform procedures (i.e., for algorithms with advice), we can simulate all randomized time- T algorithms in deterministic time $n \cdot T^{1+\epsilon}$. They also showed that, assuming $\#\text{NSETH}$ (i.e., a strong assumption from the exponential-time hypotheses family, see [7]), this is optimal: there is no worst-case derandomization running in time $n \cdot T^{1-\epsilon}$.

Unfortunately, for fast randomized algorithms (e.g., running in linear time), this overhead is significant. Is this indeed the price we must pay for derandomization?

“Free lunch” derandomization

A subsequent work of Chen and Tell [11] offered a way out: Following Impagliazzo and Wigderson [26], they considered derandomization that succeeds not in the worst-case, but over *all polynomial-time samplable distributions*. That is, the deterministic simulation errs on some inputs, but these inputs are infeasible to find. In other words, such derandomization appears correct to every efficient observer.

► **Definition 1** (heuristic simulation). *For $L \subseteq \{0,1\}^*$ and a class $\mathcal{C}$ of languages, we say that $L \in \text{heur-}\mathcal{C}$ if there is $L' \in \mathcal{C}$ such that for every probabilistic polynomial-time algorithm F it holds that $\Pr_{x \leftarrow F(1^n)}[x \in \Delta(L, L') \cap \{0,1\}^n] \leq n^{-\omega(1)}$. The definition extends to the more general notion of promise-problems $\Pi = (\Pi^Y, \Pi^N) \in \text{heur-pr}\mathcal{C}$ in the natural way (see [1]).*

Constructing free lunch derandomization algorithms is a challenging problem, and at the moment we only have one conditional construction (see below). We point out two concrete technical obstacles that have so far hindered progress:

1. **The “hybrid argument” challenge.**¹ Assume that we just want to reduce the number of coins of a probabilistic polynomial-time algorithm from $\text{poly}(n)$ to n^ϵ . Almost all known ways to do this incur a polynomial time overhead, due to the “hybrid argument” challenge, which has been studied for decades (see, e.g., [35, 29], and the references therein).

¹ This is usually referred to as the “hybrid argument barrier”, and we replace “barrier” with “challenge” to make the point that this challenge should be tackled.

The two known ways to handle this barrier either assume one-way functions [12], or assume hardness against non-uniform Merlin-Arthur circuits [17]. We are not aware of any reason to suspect that either of these assumptions is necessary to deduce the derandomization outcome (even just “morally” or qualitatively, let alone quantitatively).

2. **The lack of technical tools to exponentially reduce the number of coins (with no overhead).** Assume that we somehow handled the hybrid argument challenge, and every probabilistic polynomial-time algorithm now uses only n^ϵ random coins. Even then, when instantiating the known hardness-vs-randomness tools with standard assumptions (e.g., circuit lower bounds, or hardness for standard probabilistic algorithms), we incur significant runtime overheads. In particular, the many constructions of targeted pseudorandom generators and hitting set-generators from recent years incur such runtime overheads (see Section 2.1).

The only previously known way to bypass this falls back to using classical technical tools (i.e., it simply used the Nisan-Wigderson generator [32]), instantiated with non-standard assumptions. Specifically, the single previously known result relied on an ad-hoc assumption, which – yet again – we have no reason to believe is necessary for the conclusion.

The aforementioned previously known result, from the original work of Chen and Tell [11], deduced that $\text{BPTIME}[T] \subseteq \text{heur-DTIME}[T^{1+O(\epsilon)}]$ relying on one-way functions (to bypass the hybrid argument challenge), and on the following assumption: There is a function $f: \{0,1\}^n \rightarrow \{0,1\}^{n^\epsilon}$ such that $f(x)$ is hard to *approximately batch-compute* when x is sampled from *any polynomial-time samplable distribution*.² In an analogous result for the non-deterministic setting, a subsequent work of Chen and Tell [14] deduced free lunch derandomization of certain classes of interactive proof systems into deterministic $\mathcal{NP}$ -type protocols, under stronger assumptions of a similar “non-batch-computability” flavor (see [14] for precise details).

1.1 Our contribution, at a high level

In this work we deduce free lunch derandomization from a variety of natural hardness assumptions. The common denominator for all of our assumptions is hardness over all polynomial-time samplable distributions, which is necessary for the conclusion (see [1]). This significantly strengthens the evidence for the possibility of free lunch derandomization, both since our assumptions are more standard and well-studied (compared to non-batch-computability), and since the conclusion is now known to follow from various different natural assumptions.

Our main technical contribution is a dedicated set of technical tools for studying the problem of free lunch derandomization, addressing the second technical obstacle among the two obstacles mentioned above. Specifically, we construct targeted pseudorandom generators that are superfast, and that completely avoid the large polynomial time overheads that were inherent to many recent constructions. Our most interesting technical contribution is an alternative construction for the targeted generator of Chen and Tell [11] (i.e., their generator

² That is, each output coordinate $f(x)_i$ is computable in time $T^{1+\epsilon}$, but for every time- T algorithm A , and every polynomial-time samplable distribution over inputs x , with all but negligible probability over x it holds that $A(x)$ does not print all of $f(x)$ in time significantly faster than $T \cdot |f(x)|$ (i.e., in time $T \cdot |f(x)|^\epsilon$). (In fact, they assumed that $A(x)$ cannot even print an approximate version of $f(x)$; we ignore this for simplicity of exposition.)

that was originally used to deduce $pr\mathcal{BPP} = pr\mathcal{P}$, rather than free lunch derandomization), which is considerably faster than the original construction, and also more natural and technically intuitive.

We view our work as the first technical step following [11] towards deducing free lunch derandomization from necessary assumptions. As part of this effort, we also show that non-batch-computability as in [11, 14] (by itself, i.e. not necessarily over all polynomial-time samplable distributions) is not rare or hard to get: we deduce it from standard worst-case hardness assumptions. This gives further evidence that the non-batch-computability assumption was ad-hoc, whereas the main key is hardness over all polynomial-time samplable distributions.

A preliminary observation about a more relaxed notion

A more relaxed notion than the one in Definition 1 was considered in several prior works, which considered derandomization that has essentially no time overhead and that succeeds on average over the *uniform distribution*. Previous works deduced such derandomization from OWFs and additional hardness assumptions (see [12, 11]). We observe that such derandomization actually follows from OWFs *alone*, without additional assumptions, as a consequence of a lemma by Klivans, van Melkebeek, and Shaltiel [27] (see [1]). Thus, throughout the paper we focus on the stronger notion of free lunch derandomization from Definition 1, wherein errors are infeasible to find.

Assuming one-way functions

As in previous works, our results assume the existence of one-way functions. Since this is a ubiquitous and widely believed assumption, it strikes us as more urgent to improve the ad-hoc non-batch-computability assumption from [11, 14] than to remove the OWF assumption. We stress that for free lunch derandomization (i.e., as in Definition 1), the standard OWFs we assume (secure against polynomial-time adversaries) do not seem to yield anything better than subexponential-time derandomization.³ Moreover, our results actually only need a weaker assumption, which distills what is actually used in OWFs; for details and further discussion, see Section 1.2.2 and [1].

1.2 Free lunch derandomization from standard hardness

Our first result obtains superfast derandomization, as well as free lunch derandomization with a small amount of advice bits (i.e., $\tilde{O}(\log n)$) from hardness over all polynomial-time samplable distributions of functions computable in *bounded depth* $n^{o(1)}$. That is:

► **Theorem 2** (free lunch derandomization from hardness of $n^{o(1)}$ -depth circuits; informal, see [1]). *Assume that OWFs exist, and that for every polynomial $T(n)$ there is $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ computable by sufficiently uniform circuits of size $T^{1+\epsilon}$ and depth $n^{o(1)}$ that is hard for probabilistic algorithms running in time $T^{1-\epsilon}$ over all polynomial-time samplable distributions. Then, for every polynomial $T(n)$ we have that $prBPTIME[T] \subseteq \text{heur-prDTIME}[T^{2+O(\epsilon)}]$, and*

$$prBPTIME[T] \subseteq \text{heur-prDTIME}[T^{1+O(\epsilon)}]/\tilde{O}(\log n) .$$

³ Alternatively, standard OWFs can yield free lunch derandomization with n^ϵ bits of advice (see Section 2.1.3).

The $\tilde{O}(\log n)$ bits of advice can be removed, at the cost of strengthening the OWFs assumption to a more general one of a similar flavor; see Section 1.2.2 for details. The crux of Theorem 2 is that we only assume standard hardness for functions computable in $n^{o(1)}$ -depth, which is a more standard and well-studied assumption than non-batch-computability.

The proof of Theorem 2 is the main technical contribution of this work. Indeed, while the assumption in Theorem 2 is reminiscent of the ones used to deduce $\text{prBPP} = \text{prP}$ in [11], we do not use their technical tools. Loosely speaking, we construct a new targeted generator based on functions computable in bounded depth such that the generator has almost no time overhead. This construction can be viewed as a variant of the generator from [11], which was also based on functions computable in bounded depth. The main new insight underlying our construction is that we can replace the doubly efficient proof systems of Goldwasser, Kalai, and Rothblum [22] (which were used in the previous construction) with PCP-based techniques, and in particular relying on arithmetizations of highly efficient sorting networks.

This construction is of independent interest (i.e., beyond free lunch derandomization). As mentioned above, it is more natural and technically intuitive than the previous one, while relying on very similar assumptions regarding the hard function. Thus, it can serve as a general alternative to the known proof-system-based targeted PRGs. See Section 2.1 for details.

1.2.1 Free lunch derandomization from two other assumptions

We are also able to deduce the same conclusion as in Theorem 2 from other natural assumptions. For example, using the new targeted generator mentioned above, we can deduce the same conclusion from the assumption that there are functions computable in time T but not in space $T^{1-\epsilon}$ and time $T^{1+\epsilon}$, where hardness is over all polynomial-time samplable distributions.

► **Theorem 3** (free lunch derandomization from time-space tradeoffs; informal, see [1]). *Assume that OWFs exist, and that for every polynomial $T(n)$ there is $f: \{0,1\}^* \rightarrow \{0,1\}^*$ computable in time T that is hard for probabilistic algorithms running in space $T^{1-\epsilon}$ and time $T^{1+\epsilon}$ over all polynomial-time samplable distributions. Then, for every polynomial $T(n)$ we have $\text{prBPTIME}[T] \subseteq \text{heur-prDTIME}[T^{2+O(\epsilon)}]$, and $\text{prBPTIME}[T] \subseteq \text{heur-prDTIME}[T^{1+O(\epsilon)}]/\tilde{O}(\log n)$.*

In yet another variation on the assumptions, we also show that free lunch derandomization follows from the existence of a function $x \mapsto f(x)$ such that $f(x)$ is hard to *efficiently learn* (from membership queries) when x comes from any polynomial-time samplable distribution.

► **Theorem 4** (free lunch derandomization from hardness of learning; informal, see [1]). *Assume that OWFs exist, let T be a polynomial, and assume that there is $f: \{0,1\}^n \rightarrow \{0,1\}^{k=n^\epsilon}$ computable in time $T^{1+O(\epsilon)}$ such that for every probabilistic algorithm M running in time $T^{1+\epsilon}$, when sampling x from any polynomial-time samplable distribution, with all but negligible probability M fails to learn $f(x)$ with accuracy 0.99 from $k^{0.1}$ membership queries. Then, $\text{prBPTIME}[T] \subseteq \text{heur-prDTIME}[T^{1+O(\epsilon)}]$.*

In contrast to Theorem 3, the proof of Theorem 4 does not leverage our new technical tools, and instead relies on ideas similar to the ones of Liu and Pass [30, 31], and uses the Nisan-Wigderson generator. Similarly to [30, 31], we can actually obtain an equivalence between the conclusion and the hardness hypothesis, assuming OWFs; see [1] for details.

The point in stating the variations above is to demonstrate that several different hardness assumptions suffice for free lunch derandomization, the only common feature of which is hardness over all polynomial-time samplable distributions.

1.2.2 What is actually being used in OWFs? Removing the $\tilde{O}(\log n)$ advice bits

If one is willing to generalize the OWF assumption, then the $\tilde{O}(\log n)$ bits of advice can be removed. Specifically, we introduce a natural assumption that distills the actual content of the OWF assumption that is being used in the proofs and generalizes it, and show that replacing OWFs by this more general assumption suffices to eliminate the advice.

In our proofs and in [12, 11], the OWF is only used to obtain a PRG that is computable in near-linear-time $n^{1+\epsilon}$, has polynomial stretch $n^\epsilon \mapsto n$, and fools linear-time algorithms.⁴ This is spelled out in [1]. We observe that the crucial parameters in the foregoing assumption are: (1) The near-linear running time of the PRG; (2) The lower running time of the distinguishers; (3) The polynomial stretch. What does *not* seem like a key defining property, however, is that the polynomial stretch is $n^\epsilon \mapsto n$, rather than $\ell^\epsilon \mapsto \ell$ for some $\ell \ll n$. Hence, a more general version of the foregoing assumption asserts that there is a PRG running in time $n^{1+\epsilon}$, fooling $O(n)$ -time adversaries, and stretching ℓ^ϵ bits to ℓ bits, where ℓ may be smaller than n . This general version is spelled out in [1], and may be of independent interest.

The more general assumption, by itself, does not seem to yield anything better than subexponential-time derandomization (as one may expect, given that the PRG only has polynomial stretch), and certainly does not seem to imply any cryptographic primitive. However, we prove that if we replace OWFs by this assumption in Theorems 2 and 3, we can deduce free lunch derandomization *without non-uniform advice*. For details, see Section 2.1 and [1].

1.3 Free lunch derandomization in the non-deterministic setting

In the non-deterministic setting, the situation gets quite better. In this setting, we are interested either in derandomizing probabilistic algorithms non-deterministically (i.e., in showing superfast versions of $\mathcal{BPP} \subseteq \mathcal{NP}$) or in derandomizing Merlin-Arthur protocols non-deterministically (i.e., superfast versions of $\mathcal{MA} \subseteq \mathcal{NP}$). For concreteness, let us focus on the latter.

For context, recall that any worst-case derandomization of $\mathcal{MA}$ incurs quadratic overhead, assuming $\#\text{NSETH}$ (i.e., $\mathcal{MATIME}[T] \not\subseteq \mathcal{NTIME}[T^{2-\epsilon}]$ for any $\epsilon > 0$; see [14, Theorem 6.1]). Hence, in this context too we focus on derandomization in which errors may exist but are infeasible to find. Specifically, recalling that derandomization of an $\mathcal{MA}$ verifier is conducted with respect to both an input x and a witness π (that are given to the verifier), we are interested in a notion in which it is infeasible to find a pair (x, π) that causes a derandomization error.

Following [14], we consider free lunch derandomization of $\mathcal{MA}$ into computationally sound $\mathcal{NP}$ protocols. A $\text{cs-}\mathcal{NTIME}[T]$ protocol consists of a *deterministic* time- T verifier V and an efficient prover P such that P can prove every correct statement to V (i.e., by sending a static, $\mathcal{NP}$ -style witness π), yet no efficient uniform adversary can find an input x and proof π that mislead V , except with negligible probability.⁵ Indeed, a $\text{cs-}\mathcal{NP}$ protocol is a deterministic argument system; see [1] for formal details, and for further background on this class see [14] and the very recent work of Chen, Rothblum, and Tell [10].

⁴ We stress that the “cryptographic” properties of this PRG are not used, in the sense that we only need the PRG to fool algorithms that run in less time than the PRG itself.

⁵ When considering $\text{cs-}\mathcal{NP}$ protocols for $L \in \mathcal{MA}$, we equip the honest prover with a witness in a witness-relation for L (otherwise the prover cannot be efficient). This is the standard practice when defining argument systems in cryptography (see, e.g., [20, Section 4.8]), and it extends to deterministic argument systems; see [10].

The basic result: Free lunch derandomization from hardness in $\mathcal{FP}$

Our first result is free lunch derandomization of $\mathcal{MA}$ into $\text{cs-}\mathcal{NP}$ relying on a surprisingly clean assumption: we just need a function computable in time $n^{1+O(\epsilon)}$ that is hard for $\text{MATIME}[n^{1+\epsilon}]$ over all polynomial-time samplable distributions. Indeed, this assumption does not involve non-batch-computability, low-depth, time-space tradeoffs, or any other non-standard property.

► **Theorem 5** (free lunch derandomization of $\mathcal{MA}$; informal, see [1]). *Assume that OWFs exist, and that there is $f: \{0,1\}^n \rightarrow \{0,1\}^{n^{o(1)}}$ computable in time $n^{1+O(\epsilon)}$ that is hard for $\text{MATIME}[n^{1+\epsilon}]$ over all polynomial-time samplable distributions. Then, for every polynomial $T(n)$,*

$$\text{MATIME}[T] \subseteq \text{cs-}\mathcal{NTIME}[T^{1+\epsilon}]/\tilde{O}(\log n) .$$

The hypothesis in Theorem 5 is reminiscent of the one in [17], but they deduce worst-case derandomization with quadratic overhead (of probabilistic algorithms), whereas Theorem 5 deduces free lunch derandomization with essentially no overhead (of $\mathcal{MA}$ into $\text{cs-}\mathcal{NP}$ protocols). Indeed, we do not use their technical tools. We prove Theorem 5 using a superfast targeted generator that is suitable for the non-deterministic setting, and which is a variant of a PCP-based generator recently introduced by van Melkebeek and Sdroievski [38]. For details, see Section 2.2. Similarly to Section 1.2.2, we can remove the $\tilde{O}(\log n)$ bits of advice by replacing the OWFs assumption with an assumption about the existence of a near-linear-time PRG for a general setting of parameters (see [1] for details).

The conclusion in Theorem 5 addresses a significantly more general setting compared to prior works concerning superfast derandomization into $\text{cs-}\mathcal{NP}$ protocols. This is since prior works studied simulating subclasses of prBPP , rather than simulating all of $\mathcal{MA}$. Specifically, in [14] they deduced free lunch derandomization of doubly efficient interactive proof systems into $\text{cs-}\mathcal{NP}$ protocols (and relied on a non-batch-computability and on the classical Nisan-Wigderson generator). And in [10], they simulated uniform- $\mathcal{NC}$ by $\text{cs-}\mathcal{NP}$ protocols with a near-linear-time verifier (and relied on hardness for polylogarithmic space).⁶

A refined version: Free lunch derandomization from non-deterministic hardness

Note that in Theorem 5, the hard function is computable in $\mathcal{FP}$ yet is hard for smaller $\mathcal{MA}$ time. This was stated only for simplicity (and such an assumption also suffices to derandomize BPTIME into heur-DTIME). As one might expect, our more refined technical result relies on a non-deterministic upper bound and on a corresponding non-deterministic lower bound.

Loosely speaking, we deduce the conclusion of Theorem 5 from the existence of a function computable in $\text{cs-}\mathcal{NTIME}[n^{1+O(\epsilon)}]$ that is hard for $\text{cs-MATIME}[n^{1+\epsilon}]$ over all polynomial-time samplable distributions. That is, the upper bound and the lower bound are both non-deterministic, and both require only computational soundness. For formal definitions and a discussion of the assumption, see Section 2.2 and [1].

The proof of this result relies on an interesting technical contribution. Specifically, we construct a superfast targeted generator that is suitable for the non-deterministic setting, and that has properties useful for working with *protocols that have computational soundness* (e.g.,

⁶ The conclusions in both works are incomparable to the conclusion in Theorem 5. This is since in [14], the honest prover does not receive a witness in a witness-relation for the problem; whereas in [10] the $\text{cs-}\mathcal{NP}$ verifier runs in near-linear time even if the uniform $\mathcal{NC}$ circuit is of large polynomial size. We also note that the main result in [10] simulates a huge class (i.e., $\mathcal{PSPACE}$) by $\text{cs-}\mathcal{NP}$ protocols; this is again incomparable to Theorem 5, since the latter result focuses on tight time bounds (i.e., free lunch derandomization) whereas the former does not.

cs- $\mathcal{NP}$ protocols). The construction relies on a near-linear PCP that has an efficient *proof of knowledge*, and specifically on the construction of Ben-Sasson et al. [5]. See Section 2.2 for details.

1.4 Non-batch-computability from worst-case hardness

As another indication that the non-batch-computability assumption in [11, 14] is a red herring (or rather, an ad-hoc feature that can be replaced by various others), we prove that non-batch-computability over the *uniform distribution* follows from standard worst-case hardness assumptions. Indeed, this should not be surprising, since non-batch-computability is reminiscent of hardness of direct product (i.e., it is harder to compute k instances of a function $f(x_1), \dots, f(x_k)$ than to compute one instance $f(x)$). However, direct product hardness is known for limited models, whereas we are interested in hardness for general models (see, e.g., [33]).

Loosely speaking, we prove a general result asserting that if a function f is downward self-reducible and efficiently arithmetizable (i.e., admits an efficient low-degree extension), then hardness of f implies non-batch-computability of a related function f' (see [1]). The point is that many natural functions have these properties, and thus hardness for any of these functions implies the existence of non-batch-computable functions. For example, consider the k -Orthogonal Vectors problem (k -OV), in which we are given k sets $A_1, \dots, A_k \subseteq \{0, 1\}^d$ with $|A_i| = n$, and we want to decide whether there are $a_1 \in A_1, \dots, a_k \in A_k$ such that $a_1 \cdot \dots \cdot a_k = 0$ (this problem is a cornerstone of fine-grained complexity; see, e.g., [40, 41]). Then:

► **Theorem 6** (non-batch-computability from worst-case hardness; informal, see [1]). *If k -OV cannot be solved in randomized time $n^{k-o(1)}$, then for some $\epsilon, \delta > 0$ there is a function $f: \{0, 1\}^n \rightarrow \{0, 1\}^{r=n^\delta}$ such that:*

1. *Each output bit $f(x)_i$ can be computed in time $T(n) = \tilde{O}(n^k)$.*
2. *For every probabilistic algorithm A running in time $T \cdot r^\epsilon$, with probability 0.99 over $x \in \{0, 1\}^n$ it holds that $\Pr_{A, i \in [r]}[A(x) = f(x)_i] < 0.99$.*

The reduction that we show of worst-case computation to approximate-batch-computing is based on a direct product theorem for computing low-degree polynomials, which is targeted specifically for the setting of a k -wise direct product for a small k . See Section 2.3 for details.

2 Technical overview

Free lunch derandomization relies on targeted pseudorandom generators (targeted PRGs), as introduced by Goldreich [21].⁷ Targeted PRGs get input x and produce pseudorandomness for uniform algorithms that also get access to the same input x .

In recent years, three types of constructions of targeted PRGs have been developed: Targeted PRGs based on the classical Nisan-Wigderson PRG [32] (see, e.g., [11, Section 2.1], or [30, 31]); targeted PRGs based on the doubly efficient interactive proof system of Goldwasser, Kalai and Rothblum [22], introduced by Chen and Tell [11] (see, e.g., [8, 14, 15,

⁷ Recall that for free lunch derandomization we cannot rely on standard PRGs for non-uniform circuits. First, a PRG for size- T circuits must have seed length at least $\log(T)$, in which case enumerating over seeds (and evaluating a T -time algorithm) yields derandomization in quadratic time. Secondly, such a PRG necessitates circuit lower bounds, and we want constructions from hardness for uniform algorithms.

18, 29]); and targeted PRGs suitable for non-deterministic derandomization, a-la $\mathcal{MA} \subseteq \mathcal{NP}$, which are based on PCPs and were introduced by van Melkebeek and Sdroievski [38] (see also [39]). For a survey see [13].

The point is that none of the constructions above suffice to get free lunch derandomization from our assumptions. Generators based on NW can be very fast, but they rely on assumptions such as non-batch-computability; generators based on interactive proofs are slow, and this obstacle seems inherently difficult to bypass (see Section 2.1); and known PCP-based generators for the non-deterministic setting lack specific features that we need in our setting (see Section 2.2). We will thus have to develop new targeted PRGs, which are fast and rely on standard assumptions, and then use additional ideas to leverage them and obtain our results.

Organization

In Section 2.1 we describe the construction underlying the proof of Theorem 2, and in Section 2.2 we describe the construction underlying the proof of Theorem 5 and of its technical extension. In Section 2.3 we describe the proof of Theorem 6.

2.1 A superfast targeted generator, and proof of Theorem 2

We first explain why known proof-system-based generators are slow, even when using very fast proof systems, and what is our main idea for doing better. Readers who are only interested in a self-contained technical description of the new generator may skip directly to Section 2.1.2.

2.1.1 Generators based on interactive proofs, and their drawbacks

Recall that the generator of Chen and Tell [11] relies on an interactive proof system (specifically, that of [22], but let us discuss their idea more generally). For each round i of the proof system, they consider the prover strategy P_i on input x as a function of the verifier's challenges up to that point, and use the *truth-table* of P_i as a hard truth-table for a classical PRG (say, [32] or [34]). The classical PRG yields a list L_i of pseudorandom strings, and they output $\cup_i L_i$.⁸

One would hope that using superfast proof systems, wherein the prover's strategy function is computable in near-linear time (e.g., [16]), would yield a superfast generator. However, this idea faces inherent challenges. First, the generator uses the *truth-table* of P_i ; thus, even if the prover's strategy is computable in near-linear time, computing it over all possible verifier challenges (across all i 's) would take at least quadratic time. Secondly, the prover's response at round i depends not only on the verifier's challenge in round i , but also on the challenges at *previous* rounds. Thus, we need each P_i to depend on sufficiently few past responses so that the truth-table of P_i is only of super-linear size. Proof systems that we are aware of, even ones with very fast provers, yield generators with large polynomial overheads due to both problems.

Beyond these technical problems, more generally, *interactive proofs do not seem to be the right tool for the job*. To see this, observe that this generator relies on a small sequence of long, static “proofs” (i.e., the P_i 's) that are all committed to *in advance*. Indeed, this is far

⁸ The generator also relies on specific features of the P_i 's, namely that they yield a downward self-reducible sequence of codewords, but let us ignore this fact for a moment.

more similar to a PCP than to an interactive proof system. The key to our new construction is using technical tools underlying classical PCP constructions in order to replace the proof system of [22]. Specifically, we will construct a superfast targeted generator using a *highly efficient sorting network*.

2.1.2 A superfast generator based on highly efficient sorting networks

We want a generator that gets input $x \in \{0, 1\}^n$, runs in near-linear time in T , and produces pseudorandomness for $T^{.99}$ -time algorithms that get x . We will prove the following:

► **Theorem 7** (superfast targeted generator; informal, see [1]). *Let $\{C_n\}$ be a sufficiently uniform circuit family of size T and depth d . Then, for every sufficiently small constant $\delta \in (0, 1)$ there is a deterministic algorithm SPRG_C and a probabilistic oracle algorithm Rec_C such that:*

1. **Generator.** *When SPRG_C gets input $x \in \{0, 1\}^n$ it runs in time $d \cdot T^{1+O(\sqrt{\delta})}$ and outputs a list of T^δ -bit strings.*
2. **Reconstruction.** *Suppose that Rec_C gets input $x \in \{0, 1\}^n$ and oracle access to a function $D: \{0, 1\}^{T^\delta} \rightarrow \{0, 1\}$ that is a $(1/T^\delta)$ -distinguisher for $\text{SPRG}_C(x)$. Then, $\text{Rec}_C(x)$ runs in time $(d + n) \cdot T^{O(\sqrt{\delta})}$, makes $T^{O(\sqrt{\delta})}$ queries to D , and with probability at least $2/3$ outputs $C_n(x)$.*

The main part in proving Theorem 7 is an encoding of the computation of $C_n(x)$ as a bootstrapping system, a-la [26, 11] (see definition below). In order to prove Theorem 7 we need an extremely efficient bootstrapping system, which is significantly faster than previously known constructions [11, 15, 8, 18, 29].

Standard setup

From here on, let $\epsilon = \Theta(\delta)$ be sufficiently small. The generator computes $C_n(x)$ and then encodes the gate-values obtained in $C_n(x)$ during the computation as a **bootstrapping system**, which is a sequence of functions $\{P_1, \dots, P_d\}$ (“layers”) with the following properties:

1. **Error-correction:** Each layer $P_i: \mathbb{F}^m \rightarrow \mathbb{F}$ is a low-degree polynomial.
2. **Base case:** Computing each entry of P_1 can be done in time $t = T^\epsilon$, given x .
3. **Downward self-reducibility (DSR):** For $i > 1$, computing each entry of P_i reduces in time t to computing entries in P_{i-1} .
4. **Faithful representation:** Computing each entry of $f(x)$ can be done in time t given P_d .

It will be crucial for us that $d' \approx d$, that each P_i is a function over a domain of size $|\mathbb{F}|^m \approx T$, and that the generator can compute the bootstrapping system from x in near-linear time in T .

Warm-up: A nicely arranged grid

Observe that the gate-values of $C_n(x)$ are already arranged into d layers $p_1, \dots, p_d: [T] \rightarrow \{0, 1\}$ with built-in DSR.⁹ For convenience, let us replicate each gate to “left” and “right” copies; that is, for every $i \in [d]$ there are now $2T$ gates in layer p_i , indexed by $(g, b) \in [T] \times \{\text{lt}, \text{rt}\}$ such that $p_i(g, \text{lt}) = p_i(g, \text{rt})$. Also, let us arithmetize the input layer p_1 in the

⁹ Indeed, we will assume that the circuit is sufficiently uniform, and in particular that given $(i, j) \in [d] \times [T]$ we can output the indices of gates in layer $i - 1$ that feed into gate j at layer i , in time $\ll T$.

standard way: Using a set $H \subseteq \mathbb{F}$ of size $|H| = T^\epsilon$ and m such that $|H|^m \geq 2T$, we define $P_1: \mathbb{F}^m \rightarrow \mathbb{F}$ to be a low-degree polynomial (i.e., of individual degree $h - 1 \approx T^\epsilon$) such that if $w_{j,b}^\rightarrow$ is the $(j, b)^{th}$ element of H^m , then $P_1(w_{j,b}^\rightarrow) = p_1(j, b)$. Since P_1 is, essentially, just a low-degree extension of $x \in \{0, 1\}^n$, it is computable at any input in time $\tilde{O}(n) \leq \tilde{O}(T)$.

Now, imagine for a moment that the circuit is a $d \times 2T$ grid such that every gate takes its inputs from the pair of gates *directly* below it. Formally, for every $(g, b) \in [T] \times \{\text{lt}, \text{rt}\}$, the value of $p_{i+1}(g, b)$ depends on $p_i(g, \text{lt})$ and $p_i(g, \text{rt})$. Then, constructing a bootstrapping system is much easier. Specifically, starting from $i = 1$ and working our way up, we have a low-degree polynomial P_i for p_i , and we can easily construct a polynomial P_{i+1} for p_{i+1} :

$$P_{i+1}(w_{j,b}^\rightarrow) = B_{i+1}(P_i(w_{j,\text{lt}}^\rightarrow), P_i(w_{j,\text{rt}}^\rightarrow)) ,$$

where B_{i+1} is a low-degree arithmetization of the Boolean gate operation in layer $i + 1$. Note that P_{i+1} defined above is a low-degree polynomial that agrees with p_{i+1} on $H^m \equiv [2T]$, and that the resulting sequence $P_1, \dots, P_d$ is downward self-reducible in the straightforward way.

Even in this easy setting we are not done, since when naively propagating this construction up across layers, the degree blows up (because $\deg(P_{i+1}) = c \cdot \deg(P_i)$ for some constant $c > 1$ that depends on the gate operation). However, we can maintain individual degree at most $h - 1$, using the linearization idea of Shen [36]. Specifically, after each P_i (and before P_{i+1}), we will add a sequence of polynomials $P_{i,j}$ that decrease the individual degree of each variable at a time to $h - 1$, while maintaining the behavior of P_i on H^m (see [1] for an implementation). Of course, after adding this sequence, it is no longer guaranteed that each gate in p_{i+1} takes its inputs directly from the two gates below it in $P_{i,\ell}$ (where $P_{i,\ell}$ is the last polynomial in the degree-reduction sequence). But observe that *if this property would hold, we would be done*.

The key component: Simple sorting

The crux of our proof is *sorting* the gate-values at each layer such that after sorting, the value of each gate g will be directly below the gates to which g feeds in the layer above. Towards this operation, we assume that the circuit is sufficiently uniform, in the following sense: Each gate has fan-out two, and there is a uniform formula of size $\text{polylog}(T)$ that gets input $(i, g, \sigma) \in [d] \times [T] \times \{0, 1\}$ and prints the index of the σ^{th} output gate of g in layer $i + 1$ (see [1]).¹⁰ In particular, we can arithmetize this formula and obtain low-degree polynomials $OUT_{i,b}$ computing the index of the b^{th} output gate (i.e., left or right) of g . Then, for each P_i , we obtain a low-degree P'_i that maps (g, b) to $(OUT_{i,b}(g), P_i(g, b))$.

We now sort the values of P'_i such that the value that originally appeared in location (g, b) (i.e., $P_i(g, b)$) will appear in location $(OUT_{i,b}(g), b)$ after sorting, for some $b \in \{\text{lt}, \text{rt}\}$. That is, we construct a sequence of polynomials $P'_{i,1}, \dots, P'_{i,\ell}$ such that, thinking of g' as an index of a gate in layer $i + 1$, we have that $\left\{ P'_{i,\ell}(g', b) \right\}_{b \in \{\text{lt}, \text{rt}\}} = \{ P'_i(IN(g', b)) \}_{b \in \{\text{lt}, \text{rt}\}}$, where $IN(g', b)$ is index index of the b^{th} gate feeding into g' . We do this by arithmetizing the operations of a sorting procedure that runs in parallel time $\ell = \text{polylog}(T)$ (and thus we only add ℓ polynomials).

We need a sorting procedure that is arithmetizable as a sequence of polynomials that are simultaneously of low-degree and downward self-reducible; that is, each $P_{i,j}$ is of low degree, and the value of $P_{i,j}(g, b)$ depends in a simple way on a small number of locations in

¹⁰ Note that many natural functions satisfy this notion of uniformity, since the adjacency relation in circuits for these functions is typically very rigid (and we can add $\log(T)$ layers above each layer to ensure that gates have fan-out two).

$P_{i,j-1}$ whose indices are easily computable from (g, b) . This seems like a chicken-and-egg problem, since our goal in constructing a bootstrapping system to begin with is precisely to encode the computation of $C_n(x)$ into a sequence of polynomials that achieve both properties simultaneously. However, we have now reduced this problem to achieving these properties only for the specific computation of a *sorting procedure*, and we can choose which sorting procedure to work with.

The key is using a highly efficient sorting network *whose operations are as simple as possible*. Indeed, recall that sorting networks work in parallel, perform simple operations, and their circuit-structure function is rigid and simple. For our purposes, the most useful network turns out to be Batcher's [4] classical **bitonic sorter**: The non-recursive implementation of this sorting network uses functionality that is as simple as one might imagine; see [1].¹¹

There are still some details left to sort out. We need to arithmetize the operations of the bitonic sorter, to arithmetize the gate operations, and to add degree-reduction polynomials in between all operations. For full details, see [1].

From bootstrapping system to targeted generator

Lifting the bootstrapping system to a proof of Theorem 7 is standard by now (see, e.g., [11] for details). In a gist, the generator maps each layer in the bootstrapping system to a list of strings, using the NW generator; and the reconstruction uses a distinguisher to iteratively compress each layer, starting from the input layer and going up until reaching the top layer (which has the output $C_n(x)$). To compress a layer, it uses the reconstruction procedure of NW, which works in small time $T^{O(\sqrt{\delta})}$ when the output length T^δ of NW is small (as it will be in our setting; see below).

Note that the overall reconstruction uses $d \ll T$ steps, each running in time $T^{O(\sqrt{\delta})}$ and using access to a T -time distinguisher. If C_n is computable in time $\bar{T} = T^{1+O(\sqrt{\delta})}$ yet hard for time $\bar{T}^{1-\epsilon}$, we obtain a contradiction. See [1] for details.

2.1.3 Proof of Theorem 2

Let us start by derandomizing $\mathcal{RTIME}[T]$. Fix a machine M running in time T and solving a problem with one-sided error. If we instantiate Theorem 7 with a function hard over all polynomial-time samplable distribution, when x is chosen from a polynomial-time samplable distribution, will all but negligible probability there exists $s \in \text{SPRG}_C(x)$ such that $M(x, s) = 1$.

Unfortunately, this only yields *quadratic time* derandomization. Specifically, if OWFs exist we can assume wlog that M uses only T^ϵ random coins (since the OWF yields a PRG with polynomial stretch running in near-linear-time; see [1]). We instantiate Theorem 7 with a hard function computable by circuits of size $T^{1+O(\epsilon)}$ and depth $T^{O(\epsilon)}$, in which case $\text{SPRG}_C(x)$ yields $T^{1+O(\epsilon)}$ pseudorandom strings for $M(x, \cdot)$.¹² However, evaluating $M(x, \cdot)$ on each of those strings (to search for s such that $M(x, s) = 1$) takes time $T^{2+O(\epsilon)}$.

¹¹ In particular, it will be very convenient for our arithmetization that the bitonic sorter compares the values of gate-values whose indices are of the form $\vec{x}$ and $\vec{y} = \vec{x} + \vec{e}_i$ where $\vec{e}_i$ has Hamming weight 1.

¹² Here is a sketch of the standard analysis. The hard function is computable in time $\bar{T} = T^{1+O(\epsilon)}$, but hard for time $\bar{T}^{1-\epsilon}$. Note that the reconstruction runs in time $T^{O(\epsilon)}$ and makes at most $T^{O(\epsilon)}$ to its distinguisher D . We will use $D_x = M(x, \cdot)$, which runs in time T . In this case the reconstruction runs in time $T^{1+O(\epsilon)} = \bar{T}^{1-\epsilon}$. If there is a polynomial-time samplable distribution such that with noticeable probability the derandomization fails, then there is a polynomial-time samplable distribution such that with noticeable probability, the reconstruction computes the hard function too quickly. See [1].

Nevertheless, using Theorem 7 we exponentially reduced the number of random coins used by M , from T^ϵ to $(1 + O(\epsilon)) \cdot \log(T)$ (since it now suffices to choose from the list $\text{SPRG}_C(x)$), and crucially, we did so *without meaningfully increasing the running time of M* .

Free lunch derandomization with small advice

We now use a stronger property of SPRG_C . Specifically, observe that the generator computes a small number $d' = d \cdot \text{polylog}(T)$ of lists, and for every x such that the reconstruction fails, at least one of the lists is *pseudorandom* for $M(x, \cdot)$. In particular, on inputs x such that $\Pr_r[M(x, r)] \geq 1/2$ we have that $\Pr_{s \in \text{SPRG}_C(x)}[M(x, s) = 1] \geq 1/O(d')$.¹³ Our first step is to increase the density of “good” strings s in the list from $1/O(d')$ to $1 - n^{-\omega(1)}$. Naively re-sampling from the list can achieve this while increasing the number of random coins to $O(\log T)^2$, and using randomness-efficient samplers, we do this with $\tilde{O}(\log T)$ random coins, and with no significant increase to the running time of M .

The crucial observation is that now there exists one seed $s \in \{0, 1\}^{\tilde{O}(\log T)}$ that is good for *all efficiently samplable distributions*, since there are only countably many such distributions. That is, for every fixed polynomial-time machine sampling a distribution $\mathbf{x} = \{\mathbf{x}_n\}$, note that the probability over $x \sim \mathbf{x}_n$ and $s \in \{0, 1\}^{\tilde{O}(\log n)}$ that $\Pr_r[M(x, r)] \geq 1/2$ yet $M(x, s) = 0$ is negligible. By a union-bound, the probability that this holds for at least one of the first (say) n Turing machines is also negligible. Thus, if we fix a good seed $s_n \in \{0, 1\}^{\tilde{O}(\log n)}$ as advice to the derandomization algorithm, then for every efficiently samplable distribution $\mathbf{x} = \{\mathbf{x}_n\}$ and every sufficiently large $n \in \mathbb{N}$, with all but negligible probability over $x \sim \mathbf{x}_n$ the derandomization algorithm $M(x, \text{SPRG}_C(x)_{s_n})$ outputs the correct decision in time $T^{1+O(\epsilon)}$.

Loose ends

The argument above only derandomizes $\mathcal{RP}$. However, since it uses a targeted PRG, it also works for promise-problems; in particular, the argument shows that $\text{prRTIME}[T] \subseteq \text{heur-DTIME}[T^{1+O(\epsilon)}]$. Now we can imitate the standard non-black-box reduction of derandomization of prBPP to derandomization of prRP (as in [6]), while noting that all the inputs to prRTIME considered in this reduction are explicitly computable in polynomial time. Thus, if this reduction fails with noticeable probability over $x \sim \mathbf{x}_n$, then an efficient adversary can find an input on which the derandomization of prRTIME fails, which is a contradiction. Moreover, the compositional overhead caused by this reduction is small when we focus on derandomization in near-linear time $T^{1+O(1)}$. See [1] for details.

Lastly, as mentioned in Section 1.2.2, we can eliminate the $\ell = \tilde{O}(\log n)$ bits of advice, by assuming a PRG that stretches ℓ^ϵ bits to ℓ bits in time $T^{1+O(\epsilon)}$ and fools uniform machines running in slightly smaller time. See [1] for details.

2.2 A superfast targeted generator based on PCPs with proofs of knowledge

As a warm-up, let us first prove Theorem 5. Recall that we have a function f computable in near-linear time $n^{1+O(\epsilon)}$ that is hard for smaller $\mathcal{MA}$ time $n^{1+\epsilon}$ over all polynomial-time samplable distributions, and we want to simulate $\text{MATIME}[T]$ in $\text{cs-NTIME}[T^{1+O(\epsilon)}]$.

¹³That is, the generator is actually a somewhere-PRG; a similar property of the generator of [11] was used in the past, see [9, 18], albeit for different purposes.

A bare-bones version of [39]

We use a variant of the targeted generator of van Melkebeek and Sdroievski [39]. Fix $L_0 \in \mathcal{MATIME}[T]$, decided by a verifier V_0 . The generator is given $x \in \{0, 1\}^n$, and it guesses a witness w for V_0 . It also guesses $f(x, w)$, and a PCP witness π for the language $L = \{((x, w), f(x, w))\}$, which is decidable in time $|((x, w), f(x, w))|^{1+O(\epsilon)} = T(|x|)^{1+O(\epsilon)}$. The generator then verifies that π is indeed a convincing witness (by enumerating the $(1+\epsilon) \cdot \log(T)$ coins of the PCP verifier), and outputs the NW PRG with π as the hard truth-table.

Our deterministic verifier V for L_0 gets x and a witness $\bar{w} = (w, f(x, w), \pi)$, uses this generator with $(x, w, f(x, w), \pi)$, and outputs $\bigwedge_{s \in \text{NW}(\pi)} V_0(x, w, s)$.¹⁴ This verifier indeed runs in time $T^{1+O(\epsilon)}$, assuming that we use a fast PCP and relying on the OWF assumption.¹⁵ Now, assume that an efficient dishonest prover $\tilde{P}$ can find, with noticeable probability, an input $x \notin L_0$ and a proof $\bar{w} = (w, f(x, w), \pi)$ such that $V(x, \bar{w}) = 1$. We show that on the fixed input (x, w) , an $\mathcal{MA}$ reconstruction procedure succeeds in computing $f(x, w)$ in time $T^{1+\epsilon}$. We stress that f is only hard over polynomial-time samplable distributions, and thus we can only use this argument to deduce that no *efficient adversary* can find x and $\bar{w}$ that mislead the verifier; that is, we derandomize $\mathcal{MATIME}[T]$ into $\text{cs-}\mathcal{NTIME}[T^{1+O(\epsilon)}]$.

How does the $\mathcal{MA}$ reconstruction work for such a fixed (x, w) ? By the above, there is π such that $D_{x,w}(r) = V_0(x, w, r)$ is a distinguisher for $\text{NW}(\pi)$. Given (x, w) as input, we run the reconstruction algorithm R_{NW} of NW. Recall that when R_{NW} gets suitable advice (which consists of random coins, and bits from the “correct” π), it uses $D_{x,w}$ to build a concise version of π . We non-deterministically guess advice for R_{NW} , and this determines a concise version of a PCP witness π' . We then guess $f(x, w)$ and run the PCP verifier on input $((x, w), f(x, w))$, giving it oracle access to π' . The point is that: (1) The running time of this entire procedure is small, and can be made $T^{1+\epsilon}$; (2) There is a guess such that this procedure accepts with probability 1, due to the perfect completeness of the PCP verifier; (3) After guessing the advice for R_{NW} , it commits to a single PCP witness π' , and thus if we guessed $f(x, w)$ incorrectly the PCP verifier will reject, with high probability. For precise details see [1].

A non-deterministic hardness assumption, and a superfast generator with witnessable soundness

Next, we would like to relax the hardness assumption, and only require that f will be computable non-deterministically. Since we are constructing a $\text{cs-}\mathcal{NP}$ protocol for $L \in \mathcal{MA}$, we need an efficient honest prover for f , and we thus use a hard function $f \in \text{cs-}\mathcal{NTIME}[n^{1+O(\epsilon)}]$.¹⁶

The $\text{cs-}\mathcal{NP}$ verifier for L is similar to the one in the “bare-bones” version above. The only difference is that now it also gets a witness w_f for f , uses a $\text{cs-}\mathcal{NP}$ -verifier V_f to compute $V_f((x, w), w_f)$ (which hopefully outputs $f(x, w)$), and applies NW to a PCP proof π_{x,w,w_f} for the $T^{1+O(\epsilon)}$ -time computable language $\{((x, w, w_f), V_f((x, w), w_f))\}$.

The main challenge is that with this new generator, the reconstruction procedure described above is not an $\mathcal{MA}$ protocol anymore. To see why, consider an efficient adversary $\tilde{P}$ that finds x and (w, w_f, π) such that $D_{x,w}(r)$ is a distinguisher for $\text{NW}(\pi_{x,w,w_f})$. The reconstruction

¹⁴Throughout the paper we assume that $\mathcal{MA}$ verifiers have perfect completeness.

¹⁵Specifically, we use a PCP in which PCP proofs for $\mathcal{DTIME}[T]$ can be computed in time $T^{1+o(1)}$. Also, relying on the OWF assumption, we can assume wlog that the $\mathcal{MA}$ verifier only uses T^ϵ random coins. Indeed, when the $\mathcal{MA}$ verifier uses T^ϵ random coins, we can instantiate NW with small output length $T^\epsilon \approx |\pi|^\epsilon$, in which case it runs in time $|\pi|^{1+O(\epsilon)} = T^{1+O(\epsilon)}$ and produces a list of size $T^{O(\epsilon)}$.

¹⁶As in any argument system, the notion of $\text{cs-}\mathcal{NP}$ for $L \in \mathcal{MA}$ is non-trivial only when the honest prover is efficient (at least when given a witness in an arbitrary $\mathcal{MA}$ -relation for L).

procedure then quickly and non-deterministically computes $V_f((x, w), w_f)$. The key problem is that V_f may err in computing f on some hard-to-find inputs. Specifically, on some inputs (x, w) , the reconstruction procedure has several possible outputs: It may be that $D_{x,w}$ is a distinguisher for $\text{NW}(\pi_{x,w,w_f})$ such that $V_f((x, w), w_f) = f(x, w)$, and also for $\text{NW}(\pi_{x,w,w'_f})$ such that $V_f((x, w), w'_f) \neq f(x, w)$; in this case, on different non-deterministic guesses the reconstruction procedure may output either $V_f((x, w), w_f)$ or $V_f((x, w), w'_f)$.

The issue above seems inherent to the common techniques in hardness-vs-randomness (see [1] for an explanation). Thus, as explained in Section 1.3, we will rely on a hardness assumption for stronger $\mathcal{MA}$ -type reconstruction procedures, in which misleading input-proof pairs do exist but are infeasible to find (i.e., for cs-MA protocols; see below).

It is still unclear why the protocol above should have such computational soundness – after all, maybe an adversary can indeed find a witness for the reconstruction (i.e., non-deterministic guesses for the protocol, and in particular a PCP witness π_{x,w,w'_f}) that cause the reconstruction to output an incorrect value (i.e., output $V_f((x, w), w'_f) \neq f(x, w)$). To ensure that no efficient adversary can do this, we construct the following superfast targeted generator, which has additional properties. The primary one is “witnessable soundness”: A witnessing algorithm can efficiently map convincing witnesses for the reconstruction procedure into convincing witnesses for V_f . Thus, since misleading input-witness pairs for V_f are infeasible to find, misleading input-witness pairs for the reconstruction of the new generator are also infeasible to find.

► **Theorem 8** (superfast targeted PRG whose reconstruction has witnessable soundness; informal, see [1]). *Consider V_f that gets (z, w') of length $N^{1+O(\epsilon)}$ and runs in linear time $N^{1+O(\epsilon)}$. For every $\delta > 0$ there is a deterministic NGen_V and a probabilistic NRec_V that satisfy the following:*

1. **Generator.** *When NGen_V gets (z, w') it runs in time $N^{1+O(\epsilon+\sqrt{\delta})}$, and if $V_f(z, w')$ does not reject, it prints a list of N^δ -bit strings (otherwise, it outputs $\perp$).*
2. **Reconstruction.** *The reconstruction NRec_{V_f} gets input z and oracle access to $D: \{0, 1\}^{N^\delta} \rightarrow \{0, 1\}$, runs in time $N^{1+\epsilon}$, guesses a witness π' , tosses random coins r , makes at most $N^{O(\sqrt{\delta})}$ oracle queries, and satisfies the following.*
 - a. **(Efficient honest prover.)** *There is a ppt oracle machine Prv_{V_f} such that for every (z, w') such that $\text{NGen}_{V_f}(z, w') \neq \perp$, and every $(1/M)$ -distinguisher D for $\text{NGen}_{V_f}(z, w')$, the probability that $\text{Prv}_{V_f}^D$ convinces NRec^D to output $V_f(z, w')$ is at least $2/3$.¹⁷*
 - b. **(Witnessable soundness.)** *There is a ppt oracle machine Wit_{V_f} satisfying the following. For any D and any π' , if $\Pr_r[\text{NRec}_{V_f}^D(z, r, \pi') = y] \geq 1/2$ for some $y \in \{0, 1\}^*$, then with probability at least $2/3$ the algorithm $\text{Wit}_{V_f}(z, \pi')$ outputs w' such that $V_f(z, w') = y$.*

The construction of Theorem 8 is based on PCPs with proofs of knowledge, as defined by Ben-Sasson et al. [5]. This notion asserts that convincing PCP witnesses can be efficiently “translated back” to satisfying assignments for the original predicate that the PCP proves. We will use the specific construction from [5], since we crucially need a PCP that is both very fast (i.e., has short witnesses that can be computed by a near-linear-time prover) and that has a polynomial-time proof of knowledge. The details appear in [1].

¹⁷That is, with probability at least $2/3$, the output π' of $\text{Prv}_{V_f}^D(z, w')$ satisfies $\Pr_r[\text{NRec}_{V_f}^D(z, r, \pi') = V_f(z, w')] = 1$.

The derandomization result itself is stated in [1]. The specific assumption that it relies on is function $f \in \text{cs-}\mathcal{NTIME}[n^{1+O(\epsilon)}, \text{poly}(n)]$ that is hard for $\text{cs-MATIME}[n^{1+\epsilon}, n^2]$ protocols over all polynomial-time samplable distributions, where the second quantitative term in both expressions denotes the runtime of the honest prover in the protocol.¹⁸ That is, for every $\text{cs-MATIME}[n^{1+\epsilon}, n^2]$ protocol with a verifier V and an honest prover P such that the protocol is computationally sound, it is infeasible to find inputs z on which P manages to convince V of the correct value of $f(z)$. We make this notion quantitatively precise in [1]. The rationale behind the hardness assumption is the obvious one: If we disregard randomness, the verifier in the upper bound has more power than the verifier in the lower bound. As an additional sanity check, a random function with (say) n^ϵ output bits also attains this hardness; and indeed, it is even plausible that there is a function in $\mathcal{FP}$ (rather than only $\text{cs-}\mathcal{NP}$) with such hardness.

2.3 Worst-case to approximate-batch-case reductions for natural functions

Informally, a function $g : \{0, 1\}^n \rightarrow \{0, 1\}^k$ is non-batch computable if any individual output bit $g(x)_i$ can be computed in time T , but no algorithm running in time significantly faster than $T \cdot k$ can correctly print all of the k output bits $g(x)$ (or a large fraction of them).

We prove that non-batch-computability assumption follows from any *worst-case* hard decision problem f that admits two natural properties:

1. An efficient low-degree extension: There is a low-degree multivariate polynomial p that computes f on binary-valued inputs ($\forall x \in \{0, 1\}^n, f(x) = 0 \iff p(x) = 0$) and is computable in time proportional to f . We denote $d = \deg(p)$.
2. Downward-self-reducibility: Solving a single instance of the problem efficiently reduces to solving many smaller instances.

Given such a hard function, we show that the k -wise direct product of p is non-batch-computable; that is, no algorithm can print a large fraction of the outputs significantly faster than computing each output independently. The key lemma is a direct product theorem that is akin to the ones in Ball et al. [3, 2], but focuses on the case in which the number of instances $k = n^\gamma$ is small. (In contrast, in prior work [3, 2] the number of instances $k = \text{poly}(n)$ was significantly larger the size n of each instance.)

The main idea

We show that any batch-solver fails on a small constant fraction $\delta > 0$ of the k -tuples, and later explain how to lift this to hardness over $1 - \delta$ of the k -tuples.

Let us first attempt a naive reduction and see where it fails. Assume towards a contradiction that a batch-solver B succeeds on more than $1 - \delta$ of the k -tuples. Given a (“large”) instance X to the problem, we apply downward self-reduction to obtain k' smaller instances $x_1, \dots, x_{k'}$; a correct solution to all k' smaller instances can be easily combined into a solution for X . Since the batch-solver only succeeds on average and we need to solve all k' instances

¹⁸The reason for bounding the running times of the honest provers is that we are computing a function f that does not necessarily have an underlying witness-relation. Again, this is standard when considering argument systems, and $\text{cs-}\mathcal{NP}$ systems were also defined this way in prior work (see, e.g., [20, Section 4.8] and [14, 10]). We note that in our assumption we will allow the honest prover in the cs-MA protocol to get a witness in an underlying witness-relation that can be efficiently generated; see [1] for precise details.

correctly, a natural idea is to use error-correction: We randomly sample a low-degree curve C passing through $x_1, \dots, x_{k'}$, apply the batch-solver on a set of $k = O(d \cdot k')$ points on this curve, and uniquely decode from $\approx \delta$ errors to obtain the unique degree- $(d \cdot k')$ polynomial $p \circ C$ that agrees with $B \circ C$.

The only problem with this idea is that the points on the curve on which we apply the batch-solver B are not uniformly distributed, and hence B is not guaranteed to succeed with probability $1 - \delta$. The main idea to resolve this is to add additional error-correction. Specifically, assume that $x_1, \dots, x_{k'}$ are embedded in points $C(1), \dots, C(k')$ on the curve, and that we decode using the points $C(k' + 1), \dots, C(k' + k)$ on the curve. For each $i \in \{k' + 1, \dots, k' + k\}$, we sample a random line L_i passing through $C(i)$ (i.e., $L_i(0) = C(i)$). Now, for each fixed $j \in [O(d)]$, consider the k -tuple that passes through the j^{th} point in each of the k lines $L_1, \dots, L_k$

$$\bar{x}_j = (L_1(j), \dots, L_k(j)) .$$

Observe that for each $j \in [d]$ the k -tuple $\bar{x}_j$ is uniformly random (over choice of C and of L_i 's), so we can apply the batch-solver B to it. In particular, by an averaging argument, with high probability over choices of C and L_i 's, for most of the points $C(i)$ (where $i \in \{k' + 1, \dots, k' + k\}$), for most of the points $j \in [d]$ we have $B(\bar{x}_j)_i = p(L_i(j))$ (i.e., $B(\bar{x}_j)$ is correct on its i^{th} coordinate).¹⁹ Thus, we now apply the Reed-Solomon unique decoder twice:

1. First, for every $i \in \{k' + 1, \dots, k\}$ we run the batch-solver B on $\bar{x}_1, \dots, \bar{x}_{O(d)}$ and look at its i^{th} coordinate. This yields a sequence of $O(d)$ values, and we uniquely decode the results, hoping to obtain the degree- d polynomial $p \circ L_i$ and evaluate it on 0 to get $p(C(i))$.
2. Secondly, analogously to the naive attempt, we now uniquely decode the sequence of k values obtained in the first step, hoping to obtain the degree- $(d \cdot k')$ polynomial $p \circ C$.

If indeed for most $C(i)$'s it holds that for most $j \in [d]$ we have $B(\bar{x}_j)_i = p(L_i(j))$, then for most $C(i)$'s we correctly obtain the value $p(C(i))$ in the first step, in which case the unique decoder outputs $p \circ C$ in the second step. Then we can compute $p \circ C(1), \dots, p \circ C(k')$ and combine the results $p(x_1), \dots, p(x_{k'})$ to compute the hard function at input X .

Remaining gaps

There are two remaining gaps in the proof above. First, the proof shows that every batch-solver fails on a small fraction $\delta > 0$ of the inputs, whereas we are interested in showing that every batch-solver fails on a very large fraction $1 - \delta$ of the inputs.

We bridge this gap by applying direct-product *again*, which increases the average-case hardness from δ to $1 - \delta$, carefully adapting well-known techniques from a sequence of works by Impagliazzo et al. [23, 24]. Their techniques require a way to verify that a batch-computation is correct,²⁰ and the key observation is that in our setting, we can efficiently test whether a batch-solver correctly computes $1 - \epsilon$ of the bits of $f(X)$, by sampling $O(1/\epsilon)$ output bits and computing f at each of these bits. (Observe that this does not significantly increase the running time of the batch-solver.)

¹⁹ Our actual argument will use Chebyshev's inequality (rather than an averaging argument), and to get pairwise independence we will use L_i 's that are quadratic curves. For simplicity, we hide this in the high-level overview.

²⁰ In other words, they only yield a list-decoder rather than a decoder, and we need to weed the list to find which candidate is correct.

The second gap is that the proof shows hardness of batch-computing a polynomial over a large field, rather than a Boolean function; we bridge this gap via the standard approach of applying the Hadamard code to the polynomial. For precise details, see [1].

References

- 1 Marshall Ball, Lijie Chen, and Roei Tell. Towards free lunch derandomization from necessary assumptions (and owfs). *Electronic Colloquium on Computational Complexity: ECCC*, 32:010, 2025.
- 2 Marshall Ball, Juan A. Garay, Peter Hall, Aggelos Kiayias, and Giorgos Panagiotakos. Towards permissionless consensus in the standard model via fine-grained complexity. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part II*, volume 14921 of *Lecture Notes in Computer Science*, pages 113–146. Springer, 2024. doi:10.1007/978-3-031-68379-4_4.
- 3 Marshall Ball, Alon Rosen, Manuel Sabin, and Prashant Nalini Vasudevan. Proofs of work from worst-case assumptions. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part I*, volume 10991 of *Lecture Notes in Computer Science*, pages 789–819. Springer, 2018. doi:10.1007/978-3-319-96884-1_26.
- 4 Kenneth E. Batchier. Sorting networks and their applications. In *American Federation of Information Processing Societies: AFIPS Conference Proceedings: 1968 Spring Joint Computer Conference, Atlantic City, NJ, USA, 30 April - 2 May 1968*, volume 32 of *AFIPS Conference Proceedings*, pages 307–314. Thomson Book Company, Washington D.C., 1968. doi:10.1145/1468075.1468121.
- 5 Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, and Eran Tromer. On the concrete efficiency of probabilistically-checkable proofs. In *Proc. 45th Annual ACM Symposium on Theory of Computing (STOC)*, pages 585–594, 2013. doi:10.1145/2488608.2488681.
- 6 Harry Buhrman and Lance Fortnow. One-sided versus two-sided error in probabilistic computation. In *Proc. 16th Symposium on Theoretical Aspects of Computer Science (STACS)*, pages 100–109, 1999. doi:10.1007/3-540-49116-3_9.
- 7 Marco L. Carmosino, Jiawei Gao, Russell Impagliazzo, Ivan Mihajlin, Ramamohan Paturi, and Stefan Schneider. Nondeterministic extensions of the strong exponential time hypothesis and consequences for non-reducibility. In *Proc. 7th Conference on Innovations in Theoretical Computer Science (ITCS)*, pages 261–270, 2016. doi:10.1145/2840728.2840746.
- 8 Lijie Chen, Zhenjian Lu, Igor Carboni Oliveira, Hanlin Ren, and Rahul Santhanam. Polynomial-time pseudodeterministic construction of primes, 2023. Under Submission.
- 9 Lijie Chen, Ron D. Rothblum, and Roei Tell. Unstructured hardness to average-case randomness. In *Proc. 63rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2022.
- 10 Lijie Chen, Ron D. Rothblum, and Roei Tell. Fiat-shamir in the plain model from derandomization (or: Do efficient algorithms believe that $\text{NP} = \text{PSPACE}$?). In *Proc. 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025.
- 11 Lijie Chen and Roei Tell. Hardness vs randomness, revised: Uniform, non-black-box, and instance-wise. In *Proc. 62nd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 125–136, 2021. doi:10.1109/FOCS52979.2021.00021.
- 12 Lijie Chen and Roei Tell. Simple and fast derandomization from very hard functions: Eliminating randomness at almost no cost. In *Proc. 53rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 283–291, 2021. doi:10.1145/3406325.3451059.
- 13 Lijie Chen and Roei Tell. Guest column: New ways of studying the $\text{BPP} = \text{P}$ conjecture. *ACM SIGACT News*, 54(2):44–69, 2023. doi:10.1145/3604943.3604950.

- 14 Lijie Chen and Roei Tell. When Arthur has neither random coins nor time to spare: Superfast derandomization of proof systems. In *Proc. 55th Annual ACM Symposium on Theory of Computing (STOC)*, 2023.
- 15 Lijie Chen, Roei Tell, and R. Ryan Williams. Derandomization vs refutation: A unified framework for characterizing derandomization, 2023. Under Submission.
- 16 Graham Cormode, Michael Mitzenmacher, and Justin Thaler. Practical verified computation with streaming interactive proofs. In *Proc. 3rd Conference on Innovations in Theoretical Computer Science (ITCS)*, pages 90–112, 2012. doi:10.1145/2090236.2090245.
- 17 Dean Doron, Dana Moshkovitz, Justin Oh, and David Zuckerman. Nearly optimal pseudorandomness from hardness. In *Proc. 61st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1057–1068, 2020. doi:10.1109/FOCS46700.2020.00102.
- 18 Dean Doron, Edward Pyne, and Roei Tell. Opening up the distinguisher: a hardness to randomness approach for $BPL=L$ that uses properties of BPL. In *Proc. 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 2039–2049, 2024. doi:10.1145/3618260.3649772.
- 19 Dean Doron and Roei Tell. Derandomization with minimal memory footprint. *Electronic Colloquium on Computational Complexity: ECCC*, 30:036, 2023.
- 20 Oded Goldreich. *The Foundations of Cryptography - Volume 1: Basic Techniques*. Cambridge University Press, 2001. doi:10.1017/CB09780511546891.
- 21 Oded Goldreich. In a world of $P=BPP$. In *Studies in Complexity and Cryptography. Miscellanea on the Interplay Randomness and Computation*, pages 191–232, 2011. doi:10.1007/978-3-642-22670-0_20.
- 22 Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: interactive proofs for muggles. *Journal of the ACM*, 62(4):27:1–27:64, 2015. doi:10.1145/2699436.
- 23 Russell Impagliazzo, Ragesh Jaiswal, and Valentine Kabanets. Chernoff-type direct product theorems. In Alfred Menezes, editor, *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 500–516. Springer, 2007. doi:10.1007/978-3-540-74143-5_28.
- 24 Russell Impagliazzo, Ragesh Jaiswal, Valentine Kabanets, and Avi Wigderson. Uniform direct product theorems: simplified, optimized, and derandomized. *SIAM Journal of Computing*, 39(4):1637–1665, 2010. doi:10.1137/080734030.
- 25 Russell Impagliazzo and Avi Wigderson. $P = BPP$ if E requires exponential circuits: derandomizing the XOR lemma. In *Proc. 29th Annual ACM Symposium on Theory of Computing (STOC)*, pages 220–229, 1997. doi:10.1145/258533.258590.
- 26 Russell Impagliazzo and Avi Wigderson. Randomness vs. time: De-randomization under a uniform assumption. In *Proc. 39th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 734–743, 1998. doi:10.1109/SFCS.1998.743524.
- 27 Jeff Kinne, Dieter van Melkebeek, and Ronen Shaltiel. Pseudorandom generators, typically-correct derandomization, and circuit lower bounds. *Computational Complexity*, 21(1):3–61, 2012. doi:10.1007/S00037-011-0019-Z.
- 28 Adam R. Klivans and Dieter van Melkebeek. Graph nonisomorphism has subexponential size proofs unless the polynomial-time hierarchy collapses. *SIAM J. Comput.*, 31(5):1501–1526, 2002. doi:10.1137/S0097539700389652.
- 29 Jiayu Li, Edward Pyne, and Roei Tell. Distinguishing, predicting, and certifying: On the long reach of partial notions of pseudorandomness. In *Proc. 65th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2024.
- 30 Yanyi Liu and Rafael Pass. Characterizing derandomization through hardness of Levin-Kolmogorov complexity. In *Proc. 37th Annual IEEE Conference on Computational Complexity (CCC)*, volume 234 of *LIPICs. Leibniz Int. Proc. Inform.*, pages 35:1–35:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPICs.CCC.2022.35.

- 31 Yanyi Liu and Rafael Pass. Leakage-resilient hardness v.s. randomness. *Electronic Colloquium on Computational Complexity: ECCC*, TR22-113, 2022. URL: <https://eccc.weizmann.ac.il/report/2022/113>.
- 32 Noam Nisan and Avi Wigderson. Hardness vs. randomness. *Journal of Computer and System Sciences*, 49(2):149–167, 1994. doi:10.1016/S0022-0000(05)80043-1.
- 33 Ronen Shaltiel. Towards proving strong direct product theorems. *Computational Complexity*, 12(1-2):1–22, 2003. doi:10.1007/S00037-003-0175-X.
- 34 Ronen Shaltiel and Christopher Umans. Simple extractors for all min-entropies and a new pseudorandom generator. *Journal of the ACM*, 52(2):172–216, 2005. doi:10.1145/1059513.1059516.
- 35 Ronen Shaltiel and Emanuele Viola. On hardness assumptions needed for “extreme high-end” PRGs and fast derandomization. In *Proc. 13th Conference on Innovations in Theoretical Computer Science (ITCS)*, pages 116:1–116:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.ITCS.2022.116.
- 36 Alexander Shen. $IP = PSPACE$: simplified proof. *J. ACM*, 39(4):878–880, 1992. doi:10.1145/146585.146613.
- 37 Christopher Umans. Pseudo-random generators for all hardnesses. *Journal of Computer and System Sciences*, 67(2):419–440, 2003. doi:10.1016/S0022-0000(03)00046-1.
- 38 Dieter van Melkebeek and Nicollas Moeleiro. Instance-wise hardness versus randomness tradeoffs for Arthur-Merlin protocols. In *Proc. 38th Annual IEEE Conference on Computational Complexity (CCC)*, volume 264 of *LIPIcs. Leibniz Int. Proc. Inform.*, pages 17:1–17:36. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi:10.4230/lipics.ccc.2023.17.
- 39 Dieter van Melkebeek and Nicollas Moeleiro. Leakage resilience, targeted pseudorandom generators, and mild derandomization of Arthur-Merlin protocols. In *Proc. 43rd Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, volume 284 of *LIPIcs. Leibniz Int. Proc. Inform.*, pages 29:1–29:22. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPIcs.FSTTCS.2023.29.
- 40 Virginia V. Williams. Hardness of easy problems: basing hardness on popular conjectures such as the Strong Exponential Time Hypothesis. In *Proc. 10th International Symposium on Parameterized and Exact Computation*, volume 43, pages 17–29, 2015.
- 41 Virginia Vassilevska Williams. On some fine-grained questions in algorithms and complexity. In *Proceedings of the international congress of mathematicians: Rio de janeiro 2018*, pages 3447–3487. World Scientific, 2018.